

Solving Logic Puzzles with Integer Linear Programming: Mosaic, Range, and Galaxies

Alfredo Quintella Pinto
Diplôme d'Ingénieur
ENSTA Paris
alfredo.quintella@ensta-paris.fr

Gustavo Loiola Dos Santos
Diplôme d'Ingénieur
ENSTA Paris
gustavo.loiola@ensta-paris.fr

Maria Clara Oliveira Domingos Ruas
Diplôme d'Ingénieur
ENSTA Paris
maria-clara.oliveira@ensta-paris.fr

Abstract

This project addresses the resolution of three logic puzzles — **Mosaic** (difficulty 1), **Range** (difficulty 5), and **Galaxies** (difficulty 9) — sourced from Simon Tatham’s Portable Puzzle Collection. For each puzzle, we derive an Integer Linear Programme (ILP), implement it in Julia using JuMP and CPLEX, design a random instance generator, and develop a complementary heuristic solver. For Range, two connectivity enforcement strategies are compared: a classical single-commodity flow formulation and a dynamic approach based on CPLEX lazy-constraint callbacks, with the callback approach yielding consistent 2–6× speed improvements. Galaxies, as the most complex puzzle, relies entirely on lazy-constraint callbacks to enforce region connectivity under the rotational symmetry constraint. Both CPLEX solvers find optimal solutions reliably across all tested instance sizes for Mosaic and Range; Galaxies experimental results are pending.

I. INTRODUCTION

Combinatorial logic puzzles are a natural testbed for Integer Linear Programming (ILP) solvers: they are well-defined, have clear feasibility conditions, and range in difficulty from trivially constrained to highly structured. This project focuses on three puzzles from Simon Tatham’s Portable Puzzle Collection [1]: **Mosaic** (difficulty 1), a neighbourhood-counting puzzle; **Range** (difficulty 5), a visibility-and-connectivity puzzle; and **Galaxies** (difficulty 9), a rotational-symmetry partitioning puzzle.

The project requirements ask us to, for each puzzle: (i) derive a complete ILP formulation, (ii) implement it in Julia using the JuMP modelling language with IBM ILOG CPLEX as the backend solver, (iii) build a random instance generator, (iv) implement a heuristic solver, and (v) analyse computational performance across a batch of generated instances.

Range and Galaxies both require handling global structural constraints that cannot be encoded compactly upfront. For Range, two strategies are explored and compared: embedding white-cell connectivity as a polynomial-size flow network, and enforcing it dynamically through CPLEX *lazy-constraint callbacks*. For Galaxies — the most difficult puzzle — region connectivity under the 180° rotational symmetry constraint is handled exclusively through lazy-constraint callbacks, as no compact polynomial formulation is known.

The remainder of this report is organised as follows. Section II covers Mosaic. Section III covers Range, including the two connectivity strategies and their performance comparison. Section IV covers Galaxies. Section V summarises the findings and outlines future directions.

II. MOSAIC

A. Puzzle Description

Mosaic is played on an $n \times m$ grid. Some cells contain integer clues in $\{0, \dots, 9\}$. The goal is to colour each cell either black or white such that, for every clue cell (i, j) with value $v_{i,j}$, the number of black cells in the 3×3 neighbourhood centred on (i, j) is exactly $v_{i,j}$. Cells without a clue are unconstrained. The neighbourhood is clipped to the grid boundary, so corner and edge cells have smaller neighbourhoods.

B. ILP Formulation

1) *Sets and Decision Variables:* Let $[n] = \{1, \dots, n\}$ and $[m] = \{1, \dots, m\}$ denote the row and column index sets. Let $\mathcal{C} \subseteq [n] \times [m]$ be the set of clue cells.

$$x_{i,j} \in \{0, 1\} \quad \forall i \in [n], j \in [m]$$

where $x_{i,j} = 1$ if cell (i, j) is coloured black, and $x_{i,j} = 0$ if white.

2) *Objective*: Mosaic is a pure feasibility problem; there is no cost to minimise. We set a constant objective:

$$\min 0$$

3) *Constraints*: For each clue cell $(i, j) \in \mathcal{C}$ with clue value $v_{i,j}$:

$$\sum_{k=\max(1, i-1)}^{\min(n, i+1)} \sum_{l=\max(1, j-1)}^{\min(m, j+1)} x_{k,l} = v_{i,j} \quad (1)$$

This is the complete model. The number of constraints equals $|\mathcal{C}|$, and the number of binary variables is $n \times m$. Because the constraints involve only a fixed 3×3 window per clue, the constraint matrix is very sparse, which CPLEX exploits efficiently.

C. Instance Generation

The generator works by constructing a valid solution first and then deriving a puzzle from it:

- 1) **Generate a random solution.** Each cell is independently set to black (1) with probability 0.5.
- 2) **Compute the full clue grid.** For each cell (i, j) , count the black cells in its 3×3 neighbourhood. This gives a grid where every cell has a valid clue.
- 3) **Apply a density mask.** Each clue is retained independently with probability $d \in [0, 1]$ (the *clue density* parameter). Cells whose clue is dropped become unconstrained (value -1 internally).

The resulting instance is guaranteed to have at least one valid solution (the one generated in step 1), though it may have multiple. Datasets are generated for sizes 3×3 , 5×5 , 10×10 , 15×15 , and 20×20 , with densities $d \in \{0.3, 0.5, 0.7, 0.9\}$, 10 instances each.

D. Heuristic Solver

The heuristic applies iterative *local constraint propagation*:

- 1) Initialise all cells as *unknown*.
- 2) For every clue cell (i, j) , let b be the number of known-black neighbours and u the number of unknown neighbours. Compute the *remaining demand* $r = v_{i,j} - b$.
 - If $r = u > 0$: all unknown neighbours must be black. Mark them.
 - If $r = 0$ and $u > 0$: all unknown neighbours must be white. Mark them.
- 3) Repeat step 2 until no new assignments are made.
- 4) If unknowns remain (propagation is stuck), fill each with a random $\{0, 1\}$ value and verify the full solution.

The heuristic is re-run up to a 10-second time limit, repeating the random fallback on failure. On small and well-constrained grids it converges immediately through pure propagation; on large sparse grids it falls back to random guessing.

E. Results

All experiments were run on the same machine with a 60-second CPLEX time limit per instance.

1) *CPLEX*: CPLEX finds a feasible solution on every instance without exception. Solve times are presented in Table I.

TABLE I
MOSAIC CPLEX PERFORMANCE SUMMARY.

Grid size	Mean solve time (s)	Max solve time (s)
3×3	0.02	0.10
5×5	0.02	0.04
10×10	0.02	0.04
15×15	0.02	0.05
20×20	0.04	0.38

The ILP formulation is so tightly constrained (each binary variable appears in at most nine clue constraints) that CPLEX almost always solves the LP relaxation at the root node and never needs to branch. Solve times are flat across all densities and remain well under 0.5 seconds even for 20×20 grids.

2) *Heuristic*: The heuristic solves small instances ($\leq 5 \times 5$) instantly through pure propagation. On larger instances with low clue density ($d = 0.3$), propagation typically stalls due to insufficient local information, and the random-guess fallback rarely produces a valid solution within the 10-second limit. This is an inherent limitation: without backtracking, a pure propagation heuristic cannot guarantee completeness.

3) *Discussion*: Because Mosaic has only purely local constraints, the ILP is essentially a linear system over binary variables with no global structure to exploit. CPLEX is therefore overwhelmingly superior to the heuristic for any grid larger than 5×5 . The heuristic is useful only as a fast preliminary check on small, dense instances.

III. RANGE

A. Puzzle Description

Range is played on an $n \times m$ grid. Some cells contain positive integer clues. The player must colour cells black or white (leaving clue cells white) so that all four rules hold simultaneously:

- 1) **Clue cells are white.** No numbered cell may be coloured black.
- 2) **No two adjacent black cells.** Black cells cannot share a horizontal or vertical edge.
- 3) **White connectivity.** All white cells must form a single connected region.
- 4) **Visibility.** Each clue value $v_{r,c}$ equals the total number of white cells visible from (r, c) in the four cardinal directions, counting the clue cell itself. Visibility in a direction stops at the first black cell or the grid boundary.

Rules 3 and 4 together imply no clue can equal 1 (a cell seeing only itself would be surrounded by black, isolating it). Range is rated difficulty 5 in Simon Tatham’s collection, primarily because Rule 3 (global white connectivity) requires a global constraint that cannot be encoded with a polynomial number of local inequalities without auxiliary variables.

B. ILP Formulation — Shared Constraints

1) *Sets, Indices, and Decision Variables*: Let $\mathcal{C} = \{(r, c) \mid \text{cell } (r, c) \text{ has a clue}\}$ with clue value $v_{r,c}$. Let $\mathcal{N}(i, j)$ denote the set of 4-adjacent neighbours of (i, j) within the grid. Fix any source cell $s = (s_r, s_c) \in \mathcal{C}$ (the first clue cell encountered in row-major order).

Primary binary variables:

$$b_{i,j} \in \{0, 1\} \quad \forall (i, j) \in [n] \times [m]$$

where $b_{i,j} = 1$ if cell (i, j) is black.

2) *Objective*:

$$\min 0$$

3) *Rule 1 — Clue Cells are White*:

$$b_{r,c} = 0 \quad \forall (r, c) \in \mathcal{C} \tag{R1}$$

4) *Rule 2 — No Adjacent Black Cells*:

$$\begin{aligned} b_{i,j} + b_{i,j+1} &\leq 1 & \forall i \in [n], j \in [m-1] \\ b_{i,j} + b_{i+1,j} &\leq 1 & \forall i \in [n-1], j \in [m] \end{aligned} \tag{R2}$$

5) *Rule 4 — Visibility Constraints*: For each clue cell $(r, c) \in \mathcal{C}$, we introduce auxiliary binary variables to model line-of-sight visibility. Taking the *right* direction as representative (analogous variables are defined for left, up, and down):

$$\text{vis}_{r,c,d}^R \in \{0, 1\}, \quad d = 1, \dots, m - c$$

where $\text{vis}_{r,c,d}^R = 1$ iff cell $(r, c + d)$ is visible from (r, c) looking right, i.e., all cells $(r, c + 1), \dots, (r, c + d)$ are white. This is linearised with two families of constraints:

$$\text{vis}_{r,c,d}^R \leq 1 - b_{r,c+k} \quad \forall k = 1, \dots, d \tag{R4a}$$

$$\text{vis}_{r,c,d}^R \geq 1 - \sum_{k=1}^d b_{r,c+k} \tag{R4b}$$

Constraint (R4a) ensures that if any cell on the ray is black, the visibility variable is forced to 0. Constraint (R4b) ensures that if all cells are white (the sum is 0), the variable is forced to 1 — capturing the “visible if and only if all intermediate cells are white” semantics. Symmetric constraints are added for the left ($b_{r,c-k}$), down ($b_{r+k,c}$), and up ($b_{r-k,c}$) directions.

The total visibility equation (counting the clue cell itself) must equal the clue value:

$$1 + \sum_{d=1}^{m-c} \text{vis}_{r,c,d}^R + \sum_{d=1}^{c-1} \text{vis}_{r,c,d}^L + \sum_{d=1}^{n-r} \text{vis}_{r,c,d}^D + \sum_{d=1}^{r-1} \text{vis}_{r,c,d}^U = v_{r,c} \quad (\text{R4c})$$

C. Connectivity Strategy 1: Single-Commodity Network Flow

1) *Formulation:* White connectivity (Rule 3) is modelled by routing flow from the fixed source s to every other white cell. Each white cell must receive exactly one unit of flow; black cells cannot carry flow.

We introduce continuous directed flow variables on every oriented edge of the grid:

$$f_{(i,j) \rightarrow (i',j')} \geq 0 \quad \forall (i',j') \in \mathcal{N}(i,j)$$

Flow balance at source s :

$$\sum_{(i',j') \in \mathcal{N}(s)} f_{s \rightarrow (i',j')} - \sum_{(i',j') \in \mathcal{N}(s)} f_{(i',j') \rightarrow s} = \sum_{(i,j) \neq s} (1 - b_{i,j}) \quad (\text{R3a})$$

Flow balance at every other cell $(i,j) \neq s$:

$$\sum_{(i',j') \in \mathcal{N}(i,j)} f_{(i',j') \rightarrow (i,j)} - \sum_{(i',j') \in \mathcal{N}(i,j)} f_{(i,j) \rightarrow (i',j')} = 1 - b_{i,j} \quad (\text{R3b})$$

White cells consume one unit of flow; black cells consume zero.

Flow capacity — restricted to white edges:

$$f_{(i,j) \rightarrow (i',j')} \leq M \cdot (1 - b_{i,j}), \quad f_{(i,j) \rightarrow (i',j')} \leq M \cdot (1 - b_{i',j'}) \quad (\text{R3c})$$

where $M = n \times m$ is a valid upper bound on the total number of white cells.

This formulation adds $O(nm)$ continuous variables and $O(nm)$ constraints, but introduces Big-M coefficients that weaken the LP relaxation.

D. Connectivity Strategy 2: Lazy-Constraint Callbacks

1) *Motivation:* The flow model is complete but computationally costly: the Big-M coefficients cause a weak LP relaxation, slowing CPLEX's branch-and-bound tree. An alternative is to omit connectivity constraints from the initial model and enforce them *dynamically* via lazy constraints: each time CPLEX produces an integer-feasible candidate, we check connectivity and, if violated, add a cutting plane to forbid the infeasible configuration.

2) *Separator Inequality:* Let $\hat{b}$ be an integer candidate. Run a BFS from source s through white cells ($\hat{b}_{i,j} = 0$). If a set S of white cells is not reachable from s , it forms a disconnected component. Define its outer neighbourhood:

$$N(S) = \{(i,j) \notin S \mid \exists (i',j') \in S : (i',j') \in \mathcal{N}(i,j)\}$$

In $\hat{b}$, every cell in S is white ($\hat{b} = 0$) and every cell in $N(S)$ is black ($\hat{b} = 1$). The *separator cut* forbids this exact pattern:

$$\sum_{(i,j) \in S} b_{i,j} + \sum_{(i,j) \in N(S)} (1 - b_{i,j}) \geq 1 \quad (2)$$

This forces the next solution to either colour at least one cell of S black (eliminating the isolated component) or colour at least one boundary cell of $N(S)$ white (connecting S to the rest of the grid).

3) *Callback Algorithm:* The callback is invoked by CPLEX at every integer node:

- 1) Read the candidate values $\hat{b}_{i,j}$ for all cells.
- 2) BFS from s through white cells; compute the reachable set A .
- 3) For every white cell $w \notin A$ not yet in a found component, BFS to enumerate its full component S .
- 4) Compute $N(S)$ and submit the separator cut (2) via `MOI.LazyConstraint`.
- 5) If no cut is submitted, the candidate is connected and accepted.

All disconnected components are cut in a single callback invocation, minimising round-trips to the solver. The callback is registered with `MOI.LazyConstraintCallback()`.

4) *Advantages over the Flow Model:* The callback model has no additional variables and no Big-M coefficients, yielding a tighter LP relaxation. Cuts are generated only when needed, keeping the model compact throughout branch-and-bound. In practice, Range instances are sparse enough that very few cuts are added before convergence.

E. Instance Generation

The Range generator must produce valid puzzle instances that satisfy all four rules by construction:

- 1) **Place black cells.** Shuffle all grid positions. For each position, place a black cell with probability ~ 0.15 – 0.25 , skipping if a 4-adjacent cell is already black (enforcing Rule 2).
- 2) **Check white connectivity.** Run a BFS from any white cell. If the white region is not connected (Rule 3 violated), reject and restart.
- 3) **Compute visibility clues.** For each white cell (i, j) , simulate the four visibility rays and sum the visible white cells (including (i, j) itself). Store this as the full clue grid.
- 4) **Apply density mask.** Each clue is retained independently with probability $d \in [0, 1]$. At least one clue is guaranteed.

Up to 1000 restarts are attempted per instance. Datasets are generated for sizes 5×5 , 7×7 , 10×8 , and 10×10 , with densities $d \in \{0.3, 0.5, 0.7\}$, 10 instances each.

F. Heuristic Solver

The Range heuristic applies iterative local propagation driven by the visibility constraints:

- 1) Mark all clue cells as white (Rule 1).
- 2) For each clue cell (r, c) with value $v_{r,c}$: count the white cells already guaranteed visible in each direction. If the count already equals $v_{r,c}$, the first unknown cell at the boundary of each ray must be black (placing it would exceed the count). Apply the no-adjacency check before placing black.
- 3) Apply the adjacency constraint: any unknown cell adjacent to a known black cell must be white.
- 4) Iterate steps 2–3 until no new assignments are made.
- 5) Fill remaining unknowns with white (the safest default for connectivity).
- 6) Verify the full solution with `verifySolution`.

The heuristic is attempted repeatedly within a 10-second time limit. It has no backtracking and makes no random guesses, so it either succeeds quickly or not at all.

G. Results

1) *CPLEX — Flow vs. Callback*: Both CPLEX strategies find a feasible solution on every tested instance. Table II summarises average solve times per grid size.

TABLE II
RANGE CPLEX PERFORMANCE: MEAN SOLVE TIME (SECONDS) OVER ALL DENSITIES AND INSTANCES.

Grid size	Flow (mean / max)	Callback (mean / max)
5×5	0.017 s / 0.037 s	0.003 s / 0.005 s
7×7	0.022 s / 0.040 s	0.006 s / 0.007 s
10×8	0.030 s / 0.050 s	0.014 s / 0.024 s
10×10	0.030 s / 0.046 s	0.022 s / 0.085 s

The callback strategy is consistently **2× to 6× faster** than the flow formulation across all instance classes. This improvement stems directly from the tighter LP relaxation: removing the Big-M flow variables allows CPLEX to prune branches more aggressively. Both approaches remain well within the 60-second time limit even on the largest tested instances.

Importantly, neither solve time grows significantly with instance size in our test range (5×5 to 10×10). This suggests that Range instances at these sizes are easily solvable by CPLEX, and the bottleneck is the solver overhead rather than the combinatorial complexity. Larger grids would be needed to stress-test the scalability.

2) *Heuristic*: The heuristic solves almost no instances. The only exceptions are a handful of 10×8 and 5×5 instances with high clue density ($d = 0.7$), which are solved instantly. On all other instances, the 10-second time limit is reached without finding a valid solution.

This failure mode is expected: the Range visibility constraint is non-local (a ray spans the full grid width), and without backtracking or randomisation the propagation heuristic cannot resolve conflicts arising from the global structure. Connectivity verification further constrains the search space in ways the local heuristic cannot reason about.

3) *Impact of Clue Density*: Varying the clue density between 0.3 and 0.7 does not significantly change CPLEX performance. Instances with fewer clues (low density) are structurally under-constrained and may admit many solutions, but this does not slow CPLEX since it needs only one feasible point. The heuristic is slightly more likely to succeed on high-density instances (more constraints to propagate from), but this effect is minor.

IV. GALAXIES

A. Puzzle Description

Galaxies is played on an $n \times m$ grid containing a set of dots called *galaxy centres*. The goal is to partition every cell of the grid into connected regions such that:

- 1) **One centre per region** — each region contains exactly one dot.
- 2) **180° rotational symmetry** — every region must be rotationally symmetric about its dot (i.e., for every cell (i, j) in region k with centre (c_r^k, c_c^k) , the cell $(2c_r^k - i, 2c_c^k - j)$ must also belong to region k).
- 3) **Connected regions** — each region must form a 4-connected set of cells.
- 4) **Full partition** — every cell belongs to exactly one region.

Galaxy centres can lie on cell centres, on edge midpoints, or on vertex intersections of the grid, making the symmetry axis non-integer in general. This puzzle is rated difficulty 9, the highest in the project, because the connectivity constraint (Rule 3) for each region cannot be encoded with a polynomial number of linear inequalities upfront.

B. ILP Formulation

- 1) *Sets and Decision Variables:*
- 2) *Constraints:*

C. Connectivity via Lazy-Constraint Callbacks

D. Instance Generation

E. Heuristic Solver

F. Results

V. CONCLUSION

This project demonstrated the application of Integer Linear Programming to three logic puzzles of contrasting difficulty.

Mosaic (difficulty 1) is an ideal ILP problem: all constraints are purely local, the formulation is compact, and CPLEX solves every tested instance in well under one second regardless of grid size or clue density. The constraint propagation heuristic, while fast on small grids, is unreliable on larger sparse instances due to the absence of backtracking.

Range (difficulty 5) presents a significantly richer modelling challenge. The visibility constraints required a careful linearisation through auxiliary binary variables, and white-cell connectivity required a global enforcement mechanism. Two strategies were implemented and compared:

- The **single-commodity flow** formulation encodes connectivity statically but at the cost of Big-M coefficients that weaken the LP relaxation.
- The **lazy-constraint callback** formulation avoids any extra variables upfront and enforces connectivity dynamically via separator inequalities added as CPLEX discovers integer-infeasible candidates. This yields a tighter LP relaxation and consistent 2–6× speed improvements across all instance classes.

Both CPLEX approaches solve every generated instance optimally within seconds. The Range heuristic, based on local propagation of visibility rays, is largely ineffective on random instances because Range’s constraints are inherently global and non-local.

Galaxies (difficulty 9) is the most complex puzzle tackled in this project. Rotational symmetry is encoded directly as linear equality constraints, while region connectivity — which cannot be compactly encoded — is handled exclusively via lazy-constraint callbacks. Experimental results are pending full validation of the solver pipeline.

Future work could include: (i) completing the experimental evaluation of the Galaxies solver on the generated dataset; (ii) testing larger Range instances (15×15 and beyond) to characterise scalability; and (iii) designing a stronger Range heuristic incorporating arc-consistency or constraint propagation over connectivity.

REFERENCES

- [1] S. Tatham, *Simon Tatham’s Portable Puzzle Collection*, <https://www.chiark.greenend.org.uk/~sgtatham/puzzles/>, accessed May 2026.